Google Skills

The solution: Structured output with pydantic

Enforcing JSON with output_schema

According to ADK documentation,
use Pydantic BaseModel to define the exact structure you need:

```
from pydantic import BaseModel, Field

class ProductInfo(BaseModel):
    product_name: str = Field(description="The name of the product")
    price: float = Field(description="The price in USD")
    storage: str = Field(description="The storage capacity")

structured_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="""Extract product information and respond with JSON.
    Format: {"product_name": "name", "price": 999.99, "storage": "256GB"}""",
    output_schema=ProductInfo  # Enforces this exact structure
)
```

From ADK docs:

"output_schema (optional): Define a schema representing the desired output structure.
If set, the agent's final response must be a JSON string conforming to this schema."

Reference: ADK docs - Structuring data

Core concepts

1. Defining schemas with Pydantic

Create a Pydantic model that represents your desired output:

```
from pydantic import BaseModel, Field

class CapitalOutput(BaseModel):
    capital: str =
Field(description="The capital of the
country")
```

Key points:

- Must inherit from BaseModel - you cannot use Python dictionaries or plain classes
- Each field has a type (str, float, int, bool, etc.)
- Use Field(description=...) to help the LLM understand each field
- Pass the class itself to output_schema, not an instance: output_schema=CapitalOutput

Important:

According to ADK documentation, “The input and output schema is typically a Pydantic BaseModel.” You must define your schema as a Pydantic class—dictionaries like {"name": "string"} will not work.

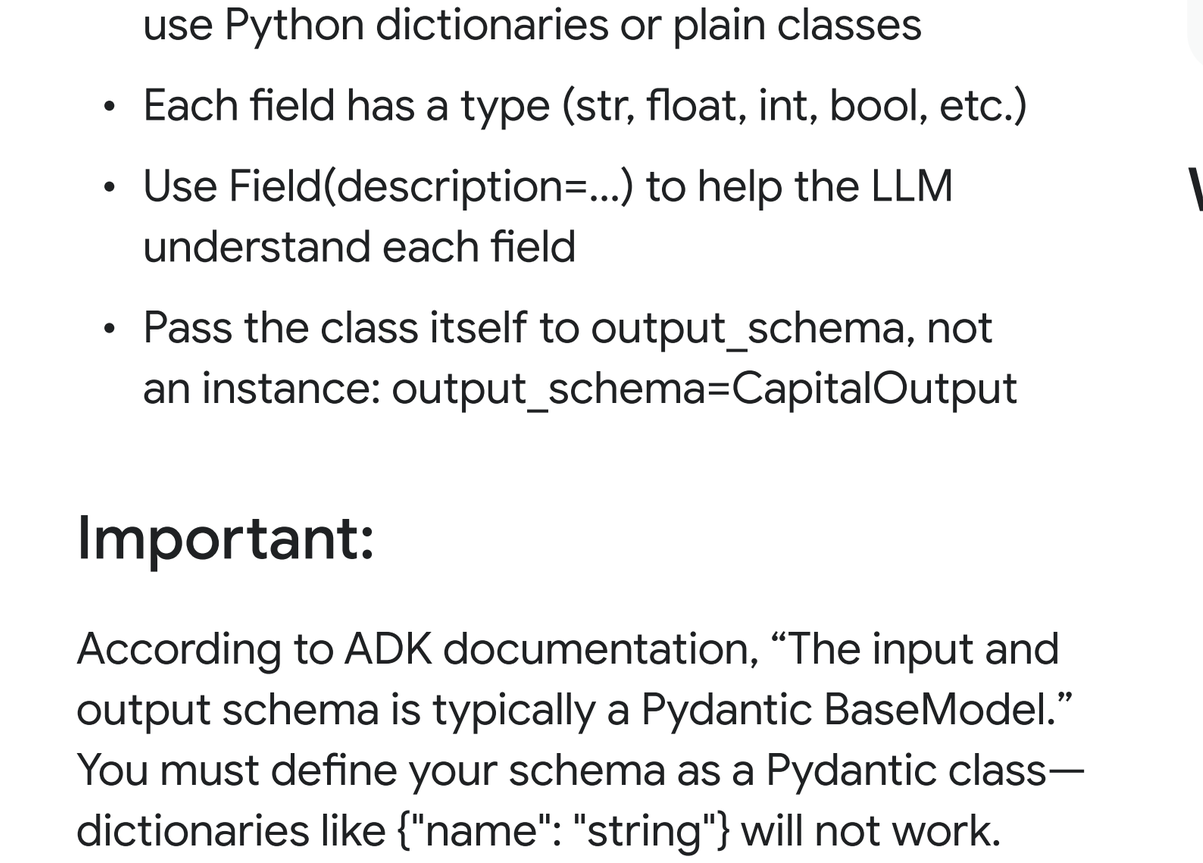
2. Using output_schema in agents

Apply the schema to your agent:

```
structured_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="capital_finder",
    instruction="""You are a Capital
Information Agent.
    Given a country, respond ONLY with
a JSON object containing the capital.
    Format: {"capital":
"capital_name"}""",
    output_schema=CapitalOutput  #
Enforce JSON output
)
```

What this does:

- The agent must return JSON matching the schema
- Invalid responses are automatically rejected
- Your code gets guaranteed structure



3. Storing Results with output_key

Save the agent's response to session state for later use:

```
structured_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Extract capital city
as JSON",
    output_schema=CapitalOutput,
    output_key="found_capital"  #
Store in
session.state["found_capital"]
)
```

Important note about multi-agent workflows:

While this module shows output_schema on the root_agent, structured output is equally valuable for intermediate sub-agents in multi-agent workflows (covered in course 5). When you have multiple agents working together, using output_schema on sub-agents ensures consistent data format when passing information between agents. For example:

```
from pydantic import BaseModel, Field
from typing import List, Dict

class ApiResponse(BaseModel):
    status: str =
Field(description="success, error, or
partial")
    status_code: int =
Field(description="HTTP status code")
    data: Dict =
Field(description="Main response
data")
    metadata: Dict =
Field(description="Request metadata")
# Must define to appear
    errors: List[str] =
Field(default=[], description="Error
messages if any")

# Without 'metadata' in the schema,
the model won't include it in output
# The schema is a contract-only
defined fields appear in responses
```

4. Complex schemas

Build more sophisticated structures:

```
from pydantic import BaseModel, Field
from typing import List, Optional

class ProductDetails(BaseModel):
    name: str =
Field(description="Product name")
    price: float =
Field(description="Price in USD")
    storage_options: List[str] =
Field(description="Available storage
capacities")
    in_stock: bool =
Field(description="Whether the
product is in stock")
    discount: Optional[float] =
Field(default=None,
description="Discount percentage if
any")

product_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="""Extract complete
product information as JSON.
    Include: name, price,
storage_options (list), in_stock
(boolean), and discount (if
mentioned).""",
    output_schema=ProductDetails
)
```

Supported types:

- Basic: str, int, float, bool
- Collections: List[T], Dict[str, T]
- Optional: Optional[T] for nullable fields
- Nested: Other BaseModel classes

Important—Schema completeness:

The schema defines the EXACT output structure. The LLM will ONLY include fields you define in your Pydantic BaseModel. If you need nested objects like metadata, errors, or pagination in your output, you must explicitly define them all in the schema:

```
from pydantic import BaseModel, Field
from typing import List, Dict

class ApiResponse(BaseModel):
    status: str =
Field(description="success, error, or
partial")
    status_code: int =
Field(description="HTTP status code")
    data: Dict =
Field(description="Main response
data")
    metadata: Dict =
Field(description="Request metadata")
# Must define to appear
    errors: List[str] =
Field(default=[], description="Error
messages if any")

# Without 'metadata' in the schema,
the model won't include it in output
# The schema is a contract-only
defined fields appear in responses
```

This ensures you get exactly the structure you need, nothing more, nothing less.

Reference: ADK Docs - Structuring Data

Hands-on example

Let's build a complete product extraction agent with structured output.

Step 1: Create the project

```
adk create product_extractor
cd product_extractor
```

Step 2: Write the agent with structured output

Replace the contents of `agent.py` with:

```
"""
Product extraction agent with structured JSON output.
Demonstrates ADK's output_schema with Pydantic BaseModel.
"""

from google.adk.agents import LlmAgent
from pydantic import BaseModel, Field

# Step 1: Define the output structure with Pydantic
class ProductInfo(BaseModel):
    product_name: str = Field(description="The full name of the product")
    price: float = Field(description="The price in USD")
    storage: str = Field(description="Storage capacity (e.g., '256GB')")
    color: str = Field(default="Not specified", description="Product color if
mentioned")

# Step 2: Create agent with output_schema
root_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="product_extractor",
    description="Extracts product information from user messages and returns
structured JSON",
    instruction="""You are a Product Information Extractor.

Your task:
- Read the user's message about a product
- Extract: product_name, price, storage, and color (if mentioned)
- Respond ONLY with valid JSON matching this format:
```

Step 2: Write the agent cont.

```
{
  "product_name": "product name here",
  "price": 999.99,
  "storage": "256GB",
  "color": "Space Black"
}

Rules:
- price must be a number (no dollar signs)
- storage must include unit (GB, TB)
- If color not mentioned, use "Not specified"
- Output ONLY the JSON, no explanation text""",
    output_schema=ProductInfo,  # Enforce this exact structure
    output_key="extracted_product"  # Store result in session state
)
```

Step 3: Run and test

```
adk web
```

Visit http://localhost:8000 and test these inputs:

Test 1: Complete information

```
You: "I want the iPhone 15 Pro with 256GB in Space Black for $999"
```

Expected JSON output:

```
{
  "product_name": "iPhone 15 Pro",
  "price": 999.0,
  "storage": "256GB",
  "color": "Space Black"
}
```

Step 3: Run and test cont.

Test 2: Missing color

```
You: "Samsung Galaxy S24 with 512GB, costs $1199"
```

Expected JSON output:

```
{
  "product_name": "Samsung Galaxy S24",
  "price": 1199.0,
  "storage": "512GB",
  "color": "Not specified"
}
```

Test 3: Different format

```
You: "Get me a MacBook Pro 1TB, price is $2499"
```

Expected JSON output:

```
{
  "product_name": "MacBook Pro",
  "price": 2499.0,
  "storage": "1TB",
  "color": "Not specified"
}
```

What to notice:

- Output is always valid JSON
- Same structure every time
- Missing fields get default values
- Easy to parse in your application

Key takeaways

- output_schema enforces structured JSON output using Pydantic BaseModel
- Pass the class (not an instance): output_schema=ProductInfo
- output_key stores results in session state for workflow integration
- Instructions must guide JSON format - tell the agent what to output
- Validation is automatic - invalid responses are rejected
- Use field descriptions to help the LLM understand each field's purpose